arXiv:1405.3866v1 [cs.CV] 15 May 2014

# Speeding up Convolutional Neural Networks with Low Rank Expansions

Max Jaderberg
max@robots.ox.ac.uk
Andrea Vedaldi
vedaldi@robots.ox.ac.uk
Andrew Zisserman
az@robots.ox.ac.uk

Visual Geometry Group
University of Oxford
Oxford, UK

**Abstract**

The focus of this paper is speeding up the evaluation of convolutional neural networks. While delivering impressive results across a range of computer vision and machine learning tasks, these networks are computationally demanding, limiting their deployability. Convolutional layers generally consume the bulk of the processing time, and so in this work we present two simple schemes for drastically speeding up these layers. This is achieved by exploiting cross-channel or filter redundancy to construct a low rank basis of filters that are rank-1 in the spatial domain. Our methods are architecture agnostic, and can be easily applied to existing CPU and GPU convolutional frameworks for tuneable speedup performance. We demonstrate this with a real world network designed for scene text character recognition, showing a possible 2.5× speedup with no loss in accuracy, and 4.5× speedup with less than 1% drop in accuracy, still achieving state-of-the-art on standard benchmarks.

## 1 Introduction

Many applications of machine learning, and most recently computer vision, have been disrupted by the use of convolutional neural networks (CNNs). The combination of an end-to-end learning system with minimal need for human design decisions, and the ability to efficiently train large and complex models, have allowed them to achieve state-of-the-art performance in a number of benchmarks [8, 10, 17, 29, 32, 36, 37]. However, these high performing CNNs come with a large computational cost due to the use of chains of several convolutional layers, often requiring implementations on GPUs [14, 17] or highly optimized distributed CPU architectures [39] to process large datasets. The increasing use of these networks for detection in sliding window approaches [9, 26, 32] and the desire to apply CNNs in real-world systems means the speed of inference becomes an important factor for applications. In this paper we introduce an easy-to-implement method for significantly speeding up pre-trained CNNs requiring minimal modifications to existing frameworks. There can be a small associated loss in performance, but this is tunable to a desired accuracy level. For example, we show that a 4.5× speedup can still give state-of-the-art performance in our example application of character recognition.

© 2014. The copyright of this document resides with its authors.
It may be distributed unchanged freely in print or electronic forms.

While a few other CNN acceleration methods exist, our **key insight is to exploit the redundancy that exists between different feature channels and filters** [6]. We contribute two approximation schemes to do so (Sect. 2) and two optimization methods for each scheme (Sect. 2.2). Both schemes are orthogonal to other architecture-specific optimizations and can be easily applied to existing CPU and GPU software. Performance is evaluated empirically in Sect. 3 and results are summarized in Sect 4.

**Related work.** There are only a few general speedup methods for CNNs. Denton *et al*. [7] use low rank approximations and clustering of filters achieving 1.6× speedup of single convolutional layers (not of the whole network) with a 1% drop in classification accuracy. Mamalet *et al*. [20] design the network to use rank-1 filters from the outset and combine them with an average pooling layer; however, the technique cannot be applied to general network designs. Vanhoucke *et al*. [39] show that 8-bit quantization of the layer weights can result in a speedup with minimal loss of accuracy. Not specific to CNNs, Rigamonti *et al*. [31] show that multiple image filters can be approximated by a shared set of separable (rank-1) filters, allowing large speedups with minimal loss in accuracy.

Moving to hardware-specific optimizations, `cuda-convnet` [17] and `Caffe` [14] show that highly optimized CPU and GPU code can give superior computational performance in CNNs. [21] performs convolutions in the Fourier domain through FFTs computed efficiently over batches of images on a GPU. Other methods from [39] show that specific CPU architectures can be taken advantage of, *e.g*. by using SSSE3 and SSSE4 fixed-point instructions and appropriate alignment of data in memory. Farabet *et al*. [8] show that using bespoke FPGA implementations of CNNs can greatly increase processing speed.

To speed up test-time in a sliding window context for a CNN, [13] shows that multi-scale features can be computed efficiently by simply convolving the CNN across a flattened multi-scale pyramid. Furthermore search space reduction techniques such as selective search [38] drastically cut down the number of times a full forward pass of the CNN must be computed by cheaply identifying a small number of candidate object locations in the image.

Note, the methods we proposed are not specific to any processing architecture and can be combined with many of the other speedup methods given above.

## 2 Filter Approximations

Filter banks are used widely in computer vision as a method of feature extraction, and when used in a convolutional manner, generate *feature maps* from input images. For an input $x \in \mathbb{R}^{H\times W}$, the set of output feature maps $Y = \{y_1, y_2, \ldots, y_N\}$, $y_n \in \mathbb{R}^{H'\times W'}$ are generated by convolving $x$ with $N$ filters $F = \{f_i\}\ \forall i \in [1\ldots N]$ such that $y_i = f_i * x$. The collection of filters $F$ can be learnt, for example, through dictionary learning methods [16, 18, 30] or CNNs, and are generally full rank and expensive to convolve with large images. Using a direct implementation of convolution, the complexity of convolving a single channel input image with a bank of $N$ 2D filters of size $d \times d$ is $\mathcal{O}(d^2NH'W')$. We next introduce our method for accelerating this computation that takes advantage of the fact that **there exists significant redundancy *between* different filters and feature channels**.

One way to exploit this redundancy is to approximate the filter set by a linear combination of a smaller basis set of $M$ filters [31, 34, 35]. The basis filter set $S = \{s_i\}\ \forall i \in [1\ldots M]$ is used to generate basis feature maps which are then linearly combined such that $y_i \simeq \sum_{k=1}^{M} a_{ik} s_k * x$. This can lead to a speedup in feature map computation as a smaller number of filters need be

convolved with the input image, and the final feature maps are composed of a cheap linear combination of these. The complexity in this case is $\mathcal{O}((d^2M+MN)H'W')$, so a speedup can be achieved if $M < \frac{d^2N}{d^2+N}$.

As shown in Rigomonti *et al.* [31], further speedups can be achieved by choosing the filters in the approximating basis to be rank-1 and so making individual convolutions *separable*. This means that each basis filter can be decomposed in to a sequence of horizontal and vertical filters $s_i * x = v_i * (h_i * x)$ where $s_i \in \mathbb{R}^{d\times d}$, $v_i \in \mathbb{R}^{d\times 1}$, and $h_i \in \mathbb{R}^{1\times d}$. Using this decomposition, the convolution of a separable filter $s_i$ can be performed in $\mathcal{O}(2dH'W')$ operations instead of $\mathcal{O}(d^2H'W')$.

The separable filters of [31] are a low-rank approximation, but performed in the *spatial* filter dimensions. Our key insight is that in CNNs substantial speedups can be achieved by also exploiting the cross-channel redundancy to perform low-rank decomposition in the *channel* dimension as well. We explore both of these low-rank approximations in the sequel.

Note that the FFT [21] could be used as an alternative speedup method to accelerate individual convolutions in combination with our low-rank cross-channel decomposition scheme. However, separable convolutions have several practical advantages: they are significantly easier to implement than a well tuned FFT implementation, particularly on GPUs; they do not require feature maps to be padded to a special size, such as a power of two as in [21]; they are far more memory efficient; and, they yield a good speedup for small image and filter sizes too (which can be common in CNNs), whilst FFT acceleration tends to be better for large filters due to the overheads incurred in computing the FFTs.

## 2.1 Approximating Convolutional Neural Network Filter Banks

CNNs are obtained by stacking multiple layers of convolutional filter banks on top of each other, followed by a non-linear response function. Each filter bank or *convolutional layer* takes an input which is a feature map $z_i(u,v)$ where $(u,v) \in \Omega_i$ are spatial coordinates and $z_i(u,v) \in \mathbb{R}^C$ contains $C$ scalar features or *channels* $z_i^c(u,v)$. The output is a new feature map $z_{i+1} \in \mathbb{R}^{H'\times W'\times N}$ such that $z_{i+1}^n = h_i(W_{in} * z_i + b_{in})\ \forall n \in [1\ldots N]$, where $W_{in}$ and $b_{in}$ denote the $n$-th filter kernel and bias respectively, and $h_i$ is a non-linear activation function such as the *Rectified Linear Unit* (ReLU) $h_i(z) = \max\{0,z\}$. Convolutional layers can be intertwined with *normalization*, *subsampling*, and *pooling layers* which build translation invariance in local neighbourhoods. Other layer types are possible as well, but generally the convolutional ones are the most expensive. The process starts with $z_1 = x$, where $x$ is the input image, and ends by, for example, connecting the last feature map to a logistic regressor in the case of classification. All the parameters of the model are jointly optimized to minimize a loss over the training set using Stochastic Gradient Descent (SGD) with back-propagation.

The $N$ filters $W_n$ learnt for each layer (for convenience we drop the layer subscript $i$) are full rank, 3D filters with the same depth as the number of channels of the input, such that $W_n(u,v) \in \mathbb{R}^C$. For example, for a 3-channel color image input, $C = 3$. The convolution $W_n * z$ of a 3D filter $W_n$ with the 3D image $z$ is the 2D image $W_n * z = \sum_{c=1}^{C} W_n^c * z^c$, where $W_n^c \in \mathbb{R}^{d\times d}$ is a single channel of the filter. This is a sum of 2D convolutions so we can think of each 3D filter as being a collection of 2D filters, whose output is collapsed to a 2D signal. However, since $N$ such 3D filters are applied to $z$, the overall output is a new 3D image with $N$ channels. This process is illustrated for the case $C = 1, N > 1$ in Fig. 1 (a). The resulting computational cost for a convolutional layer with $N$ filters of size $d \times d$ acting on $C$ input channels is $\mathcal{O}(CNd^2H'W')$.

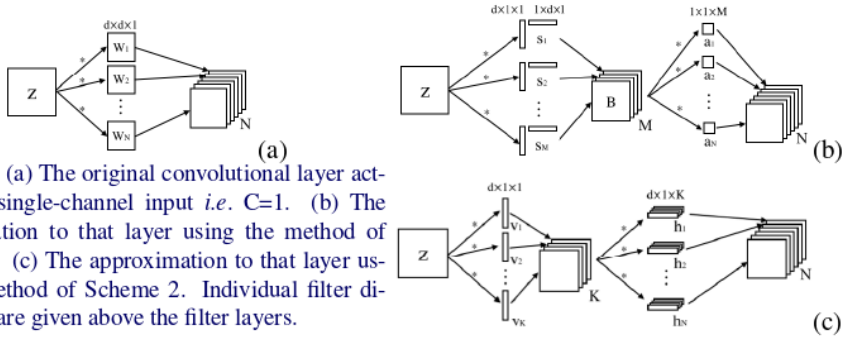



Figure 1: (a) The original convolutional layer acting on a single-channel input *i.e.* C=1. (b) The approximation to that layer using the method of Scheme 1. (c) The approximation to that layer using the method of Scheme 2. Individual filter dimensions are given above the filter layers.

We now propose two schemes to approximate a convolutional layer of a CNN to reduce the computational complexity and discuss their training in Sec. 2.2. Both schemes follow the same intuition: that CNN filter banks can be approximated using a low rank basis of filters that are separable in the spatial domain.

**Scheme 1.** The first method for speeding up convolutional layers is to directly apply the method suggested in Sect. 2 to the filters of a CNN (Fig. 1 (b)). As described above, a single convolutional layer with $N$ filters $W_n \in \mathbb{R}^{d\times d\times C}$ requires evaluating $NC$ 2D filters $F = \{W_n^c \in \mathbb{R}^{d\times d} : n \in [1\dots N], c \in [1\dots C]\}$. Note that there are $N$ filters $\{W_n^c : n \in [1\dots N]\}$ operating on each input channel $z^c$. These can be approximated as linear combinations of a basis of $M < N$ (separable) filters $S^c = \{s_m^c : m \in [1\dots M]\}$ as $W_n^c \simeq \sum_{m=1}^M a_n^{cm} s_m^c$. Since convolution is a linear operator, filter reconstruction and image convolution can be swapped, yielding the approximation $W_n * z = \sum_{c=1}^C W_n^c * z^c \simeq \sum_{c=1}^C \sum_{m=1}^M a_n^{cm}(s_m^c * z^c)$. To summarize, the direct calculation involves computing $NC$ 2D filters $W_n^c * z^c$ with cost $O(NCd^2H'W')$, while the approximation involves computing $MC$ 2D filters $s_m^c * z^c$ with cost $O(MC(d^2+N)H'W')$ – the additional $MCNH'W'$ term accounting for the need to recombine the basis response linearly. Due to the second term, the approximation is efficient only if $M \ll d^2$, *i.e.* if the number of filters in the basis is less than the filter area.

The first cost term $CMd^2H'W'$ would also suggest that efficiency requires the condition $M \ll N$; however, this can be considerably ameliorated by using *separable filters* in the basis. In this case the approximation cost is reduced to $O(MC(d+N)H'W')$; together with the former condition, Scheme 1 is then efficient if $M \ll d\min\{d,N\}$.

Note that this scheme uses $C$ filter basis $S^1, S^2, \dots, S^C$ as each operates on a different input channel. In practice, we choose $S^1 = S^2 = \dots = S^C = S$ because empirically there is no actual gain in performance and a single channel basis is simpler and more compact.

**Scheme 2.** Scheme 1 focuses on approximating 2D filters. As a consequence, each input channel $z^c$ is approximated by a particular basis of 2D separable filters. Redundancy among feature channels is exploited, but only in the sense of the $N$ *output channels*. In contrast, Scheme 2 is designed to take advantage of both input and output redundancies by considering 3D filters throughout. The idea is simple: each convolutional layer is factored as a sequence of two regular convolutional layers but with rectangular (in the spatial domain) filters, as shown in Fig. 1 (c). The first convolutional layer has $K$ filters of spatial size $d \times 1$ resulting in a filter bank $\{v_k \in \mathbb{R}^{d\times 1\times C} : k \in [1\dots K]\}$ and producing output feature maps $V$ such that $V(u,v) \in \mathbb{R}^K$. The second convolutional layer has $N$ filters of spatial size $1 \times d$ resulting in a filter bank $\{h_n \in \mathbb{R}^{1\times d\times K} : n \in [1\dots N]\}$. Differently from Scheme 1, the filters operate on multiple channels simultaneously. The rectangular shape of the filters is selected to match a

separable filter approximation. To see this, note that convolution by one of the original filters $W_n * z = \sum_{c=1}^{C} W_n^c * z^c$ is approximated by

$$W_n * z \simeq h_n * V = \sum_{k=1}^{K} h_n^k * V^k = \sum_{k=1}^{K} h_n^k * (v_k * z) = \sum_{k=1}^{K} h_n^k * \sum_{c=1}^{C} v_k^c * z^c = \sum_{c=1}^{C} \left[ \sum_{k=1}^{K} h_n^k * v_k^c \right] * z^c \quad (1)$$

which is the sum of separable filters $h_n^k * v_k^c$. The computational cost of this scheme is $O(KCdH'W)$ for the first vertical filters and $O(NKdH'W')$ for the second horizontal filter. Assuming that the image width $W \gg d$ is significantly larger than the filter size, the output image width $W \approx W'$ is about the same as the input image width $W'$. Hence the total cost can be simplified to $O(K(N+C)dH'W')$. Compared to the direct convolution cost of $O(NCd^2H'W')$, this scheme is therefore convenient provided that $K(N+C) \ll NCd$. For example, if $K$, $N$, and $C$ are of the same order, the speedup is about $d$ times.

In both schemes, we are assuming that the full rank original convolutional filter bank can be decomposed in to a linear combination of a set of separable basis filters. The difference between the schemes is how/where they model the interaction between input and output channels, which amounts to how the low rank channel space approximation is modelled. In Scheme 1 it is done with the linear combination layer, whereas with Scheme 2 the channel interaction is modelled with 3D vertical and horizontal filters inducing a summation over channels as part of the convolution.

## 2.2 Optimization

This section deals with the details on how to attain the optimal separable basis representation of a convolutional layer for the schemes. The first method (Sec. 2.2.1) aims to reconstruct the original filters directly by minimizing filter reconstruction error. The second method (Sec. 2.2.2) approximates the convolutional layer indirectly, by minimizing reconstruction error of the output of the layer.

### 2.2.1 Filter Reconstruction Optimization

The first way that we can attain the separable basis representation is to aim to minimize the reconstruction error of the original filters with our new representation.

**Scheme 1.** The separable basis can be learnt simply by minimizing the $L_2$ reconstruction error of the original filters, whilst penalizing the nuclear norm $\|s_m\|_*$ of the filters $s_m$. In fact, the nuclear norm $\|s_m\|_*$ is a proxy for the rank of $s_m \in \mathbb{R}^{d \times d}$ and rank-1 filters are separable. This yields the formulation:

$$\min_{\{s_m\},\{a_n\}} \sum_{n=1}^{N} \sum_{c=1}^{C} \left\| W_n^c - \sum_{m=1}^{M} a_n^{cm} s_m \right\|_2^2 + \lambda \sum_{m=1}^{M} \|s_m\|_*. \quad (2)$$

This minimization is biconvex, so given $s_m$ a unique $a_n$ can be found, therefore a minimum is found by alternating between optimizing $s_m$ and $a_n$. For full details of the implementation of this optimization see [31].

**Scheme 2.** The set of horizontal and vertical filters can be learnt by explicitly minimizing the $L_2$ reconstruction error of the original filters. From (1) we can see that the original filter can




(a) (b)

Figure 2: Example schematics of how to optimize separable basis approximation layers in a data reconstruction setting. (a) Approximating Conv2 with Sep2. (b) Approximating Conv3 with Sep3, incorporating the approximation of Conv2 as well.

be approximated by minimizing the objective function

$$\min_{\{h_n^k\},\{v_k^c\}} \sum_{n=1}^{N} \sum_{c=1}^{C} \left\| W_n^c - \sum_{k=1}^{K} h_n^k * v_k^c \right\|_2^2. \tag{3}$$

This optimization is simpler than for Scheme 1 due to the lack of nuclear norm constraints, which we are able to avoid by modelling the separability explicitly with two variables. We perform conjugate gradient descent, alternating between optimizing the horizontal and vertical filter sets.

### 2.2.2 Data Reconstruction Optimization

The problem with optimizing the separable basis through minimizing original filter reconstruction error is that this does not necessarily give the most optimized basis set for the end CNN prediction performance. As an alternative, one can optimize a scheme's separable basis by aiming to reconstruct the *outputs* of the original convolutional layer given training data. For example, for Scheme 2 this amounts to

$$\min_{\{h_n^k\},\{v_k^c\}} \sum_{i=1}^{|X|} \sum_{n=1}^{N} \left\| W_n * \Phi_{l-1}(x_i) - \sum_{c=1}^{C} \sum_{k=1}^{K} h_n^k * v_k^c * \Phi_{l-1}(x_i) \right\|_2^2 \tag{4}$$

where $l$ is the index of the convolutional layer to be approximated and $\Phi_l(x_i)$ is the evaluation of the CNN up to and including layer $l$ on data sample $x_i \in X$ where $X$ is the set of training examples. This optimization can be done quite elegantly by simply mirroring the CNN with the un-optimized separable basis layers, and training only the approximation layer by back-propagating the $L_2$ error between the output of the original layer and the output of the approximation layer (see Fig. 2). This is done layer by layer.

There are two main advantages of this method for optimization of the approximation schemes. The first is that the approximation is conditioned on the manifold of the training data – original filter dimensions that are not relevant or redundant in the context of the training data will by ignored by minimizing data reconstruction error, but will still be penalised by minimizing filter reconstruction error (Sec. 2.2.1) and therefore uselessly using up model capacity. Secondly, stacks of approximated layers can be learnt to incorporate the approximation error of previous layers by feeding the data through the approximated net up to layer $l$ rather than the original net up to layer $l$ (see Fig. 2 (b)). This additionally means that all the approximation layers could be optimized jointly with back-propagation.

An obvious alternative optimization strategy would be to replace the original convolutional layers with the un-optimized approximation layers and train just those layers by back-propagating the classification error of the approximated CNN. However, this does not actually result in better classification accuracy than doing $L_2$ data reconstruction optimization – in practice, optimizing the separable basis within the full network leads to overfitting of the training data, and attempts to minimize this overfitting through regularization methods like

| Layer name | Filter size | In channels | Out channels | Filters | Maxout groups | Time |
|---|---|---|---|---|---|---|
| Conv1 | $9 \times 9$ | 1 | 48 | 96 | 2 | 0.473ms (8.3%) |
| Conv2 | $9 \times 9$ | 48 | 64 | 128 | 2 | 3.008ms (52.9%) |
| Conv3 | $8 \times 8$ | 64 | 128 | 512 | 4 | 2.160ms (38.0%) |
| Conv4 | $1 \times 1$ | 128 | 37 | 148 | 4 | 0.041ms (0.7%) |
| Softmax | - | 37 | 37 | - | - | 0.004ms (0.1%) |

Table 1: The details of the layers in the CNN used with the forward pass timings of each layer.

dropout [12] lead to under-fitting, most likely due to the fact that we are already trying to heavily approximate our original filters. However, this is an area that needs to be investigated in more detail.

# 3 Experiments & Results

In this section we demonstrate the application of both proposed filter approximation schemes and show that we can achieve large speedups with a very small drop in accuracy. We use a pre-trained CNN that performs case-insensitive character classification of scene text. Character classification is an essential part of many text spotting pipelines such as [8, 9, 22, 23, 24, 25, 27, 28, 40, 42].

We first give the details of the base CNN model used for character classification which will be subject to speedup approximations. The optimization processes and how we attain the approximations of Scheme 1 & 2 to this model are given, and finally we discuss the results of the separable basis approximation methods on accuracy and inference time of the model.

**Test Model.** For scene character classification, we use a four layer CNN with a softmax output. The CNN outputs a probability distribution $p(c|x)$ over an alphabet $C$ which includes all 26 letters and 10 digits, as well as a noise/background (no-text) class, with $x$ being a grey input image patch of size $24 \times 24$ pixels, which has been zero-centred and normalized by subtracting the patch mean and dividing by the standard deviation. The non-linearity used between convolutional layers is maxout [11] which amounts to taking the maximum response over a number of linear models *e.g.* the maxout of two feature channels $z_i^1$ and $z_i^2$ is simply their pointwise maximum: $h_i(z_i(u,v)) = \max\{z_i^1(u,v), z_i^2(u,v)\}$. Table 1 gives the details of the layers for the model used, which is connected in the linear arrangement Conv1-Conv2-Conv3-Conv4-Softmax.

**Datasets & Evaluation.** The training dataset consists of 163,222 collected character samples from a number of scene text and synthesized character datasets [1, 2, 5, 15, 19, 33, 41]. The test set is the collection of 5379 cropped characters from the ICDAR 2003 training set after removing all non-alphanumeric characters as in [8, 40]. We evaluate the case-insensitive accuracy of the classifier, ignoring the background class. The Test Model achieves state-of-the-art results of 91.3% accuracy compared to the next best result of 89.8% [8].

**Implementation Details.** The CNN framework we use is the CPU implementation of `Caffe` [14], where convolutions are performed by constructing a matrix of filter windows of the input, `im2col`, and using BLAS for the matrix-matrix multiplication between the filters and data windows. We found this to be the fastest CPU CNN implementation attainable. CNN training is done with SGD with momentum of 0.9 and weight decay of 0.0005. Dropout of 0.5 is used on all layers except Conv1 to regularize the weights, and the learning rate is adaptively reduced during the course of training.

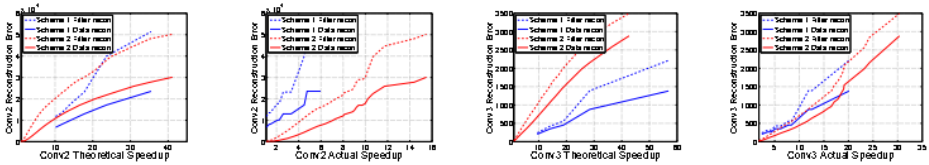



Figure 3: Reconstruction error for the theoretical and actual attained speedups on test data for Conv2 & Conv3. We do not go below 10× theoretical speedup for Scheme 1 as computation takes too long.

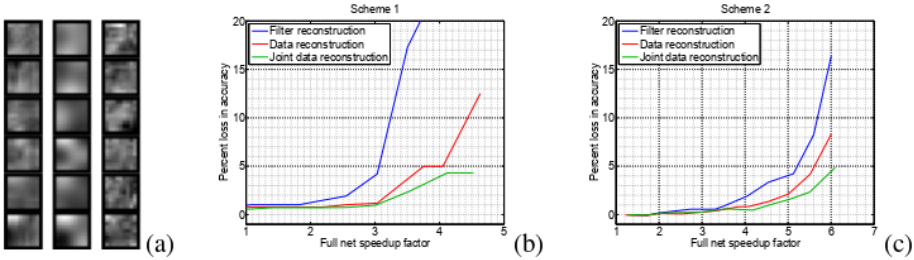


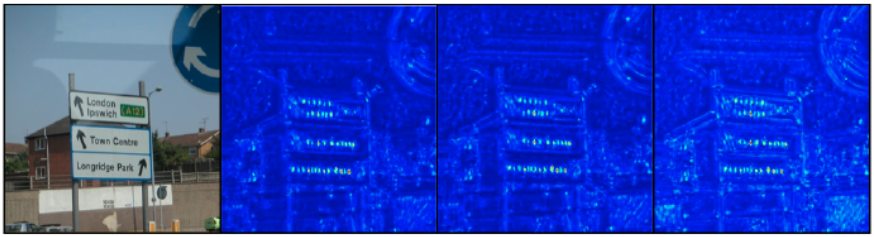

Figure 4: (a) A selection of Conv2 filters from the original CNN (left), and the reconstructed versions under Scheme 1 (centre) and Scheme 2 (right), where both schemes have the same model capacity corresponding to 10x theoretical speedup. Visually the approximated filters look very different with Scheme 1 naturally smoothing the repesentation, but both still achieve good accuracy. (b-c) The percent loss in performance as a result of the speedups attained with Scheme 1 (b) and Scheme 2 (c).

For filter reconstruction optimization, we optimize a separable basis until a stable minimum of reconstruction error is reached. For data reconstruction optimization, we optimize each approximated layer in turn, and can incorporate a fine-tuning with joint optimization.

For the CNN presented, we only approximate layers Conv2 and Conv3. This is because layer Conv4 has a $1 \times 1$ filter size and so would not benefit much from our speedup schemes. We also don't approximate Conv1 due to the fact that it acts on raw pixels from natural images – the filters in Conv1 are very different to those found in the rest of the network and experimentally we found that they cannot be approximated well by separable filters (also observed in [7]). Omitting layers Conv1 and Conv4 from the schemes does not change overall network speedup significantly, since Conv2 and Conv3 constitute 90% of the overall network processing time, as shown in Table. 1.

**Layer-wise Performance.** Fig. 3 shows the output reconstruction error of each approximated layer with the test data. It is clear that the reconstruction error worsens as the speedup achieved increases, both theoretically and practically. As the reconstruction error is that of the test data features fed through the approximated layers, as expected the data reconstruction optimization scheme gives lower errors for the same speedup compared to the filter reconstruction. This generally holds even when completely random Gaussian noise data is fed through the approximated layers – data from a completely different distribution to what the data optimization scheme has been trained on.

Looking at the theoretical speedups possible in Fig. 3, Scheme 1 gives better reconstruction error to speedup ratio, suggesting that the Scheme 1 model is perhaps better suited for approximating convolutional layers. However, when the actual measured speedups are compared, Scheme 1 is actually slower than that of Scheme 2 for the same reconstruction error. This is due to the fact that the `Caffe` convolution routine is optimized for 3D convolution (summing over channels), so Scheme 2 requires only two `im2col` and BLAS calls. However, to implement Scheme 1 with `Caffe` style convolution involving per-channel con-




Input Original CNN 3.5x speedup 6.7x speedup

Figure 5: Text spotting using the CNN character classifier. The maximum response map over the character classes of the CNN output with Scheme 2 indicates the scene text positions. The approximations have sufficient quality to locate the text, even at 6.7× speedup.

volution without channel summation, means that there are many more costly `im2col` and BLAS calls, thus slowing down the layer evaluation and negating the model approximation speedups. It is possible that using a different convolution routine with Scheme 1 will bring the actual timings closer to the theoretically achievable timings.

**Full Net Performance.** Fig. 4 (b) & (c) show the overall drop in accuracy as the speedup of the end-to-end network increases under different optimization strategies. Generally, joint data optimization of Conv2 and Conv3 improves final classification performance for a given speedup. Under Scheme 2 we can achieve a 2.5× speedup with no loss in accuracy, and a 4.5× speedup with only a drop of 1% in classification accuracy, giving 90.3% accuracy – still state-of-the-art for this benchmark. The 4.5× configuration is obtained by approximating the original 128 Conv2 filters with 31 horizontal filters followed by 128 vertical filters, and the original 512 Conv3 filters with 26 horizontal filters followed by 512 vertical filters.

This speedup is incredibly useful for sliding window schemes, allowing fast generation of, for example, detection maps such as the character detection map shown in Fig. 5. There is very little difference with even a 3.5× speedup, and when incorporated in to a full application pipeline, the speedup can be tuned to give an acceptable end pipeline result.

Comparing to an FFT based CNN [21], our method can actually give greater speedups. With the same layer setup (5×5 kernel, $16 \times 16 \times 256$ input, 384 filters), Scheme 2 gives an actual 2.4× speedup with 256 basis filters (which should result in no performance drop), compared to 2.2× in [21]. Comparing with [7], simply doing a filter reconstruction approximation with Scheme 2 of the second layer of OverFeat [32] gives a 2× theoretical speedup with only 0.5% drop in top-5 classification accuracy on ImageNet, far better than the 1.2% drop in accuracy for the same theoretical speedup reported in [7]. This accuracy should be further improved if data optimization is used.

## 4 Conclusions

In this paper we have shown that the redundancies in representation in CNN convolutional layers can be exploited by approximating a learnt full rank filter bank as combinations of a rank-1 filter basis. We presented two schemes to do this, with two optimization techniques for attaining the approximations. The resulting approximations require significantly less operations to compute, resulting in large speedups observed with a real CNN trained for scene text character recognition: a 4.5× speedup, only a drop of 1% in classification accuracy.

In future work it would be interesting to experiment with other arrangements of separable filters in layers, *e.g.* a horizontal basis layer, followed by a vertical basis layer, followed by a

linear combination layer. Looking at the filter reconstructions of the two schemes in Fig. 4 (a), it is obvious that the two presented schemes act very differently, so the connection between different approximation structures could be explored. Also it should be further investigated whether these model approximations can be effectively taken advantage of during training, with low-rank filter layers being learnt in a discriminative manner.

**Acknowledgements.** Funding for this research is provided by the EPSRC and ERC grant VisRec no. 228180.

## References

[1] http://algoval.essex.ac.uk/icdar/datasets.html.

[2] http://www.iapr-tc11.org/mediawiki/index.php/kaist_scene_text_database.

[3] O. Alsharif and J. Pineau. End-to-End Text Recognition with Hybrid HMM Maxout Models. In *International Conference on Learning Representations*, 2014.

[4] A. Bissacco, M. Cummins, Y. Netzer, and H. Neven. PhotoOCR: Reading text in uncontrolled conditions. In *International Conference of Computer Vision*, 2013.

[5] T. de Campos, B. R. Babu, and M. Varma. Character recognition in natural images. 2009.

[6] M. Denil, B. Shakibi, L. Dinh, and N. de Freitas. Predicting parameters in deep learning. In *Advances in Neural Information Processing Systems*, pages 2148–2156, 2013.

[7] E. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus. Exploiting linear structure within convolutional networks for efficient evaluation. *arXiv preprint arXiv:1404.0736*, 2014.

[8] C. Farabet, Y. LeCun, K. Kavukcuoglu, E. Culurciello, B. Martini, P. Akselrod, and S. Talay. Large-scale fpga-based convolutional networks. *Machine Learning on Very Large Data Sets*, 2011.

[9] C. Farabet, C. Couprie, L. Najman, and Y. LeCun. Scene parsing with multiscale feature learning, purity trees, and optimal covers. *arXiv preprint arXiv:1202.2160*, 2012.

[10] I. J. Goodfellow, Y. Bulatov, J. Ibarz, S. Arnoud, and V. Shet. Multi-digit number recognition from street view imagery using deep convolutional neural networks. In *International Conference on Learning Representations*, 2013.

[11] I. J. Goodfellow, D. Warde-Farley, M. Mirza, A. Courville, and Y. Bengio. Maxout networks. *arXiv preprint arXiv:1302.4389*, 2013.

[12] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving neural networks by preventing co-adaptation of feature detectors. *arXiv preprint arXiv:1207.0580*, 2012.

[13] F. Iandola, M. Moskewicz, S. Karayev, R. Girshick, T. Darrell, and K. Keutzer. Densenet: Implementing efficient convnet descriptor pyramids. *arXiv preprint arXiv:1404.1869*, 2014.

[14] Y. Jia. Caffe: An open source convolutional architecture for fast feature embedding. http://caffe.berkeleyvision.org/, 2013.

[15] D. Karatzas, F. Shafait, S. Uchida, M. Iwamura, S. R. Mestre, J. Mas, D. F. Mota, J. Almazan, L. P. de las Heras, et al. ICDAR 2013 robust reading competition. In *Document Analysis and Recognition (ICDAR), 2013 12th International Conference on*, pages 1484–1493. IEEE, 2013.

[16] K. Kavukcuoglu, P. Sermanet, Y. Boureau, K. Gregor, M. Mathieu, and Y. LeCun. Learning convolutional feature hierarchies for visual recognition. In *NIPS*, volume 1, page 5, 2010.

[17] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *NIPS*, volume 1, page 4, 2012.

[18] H. Lee, R. Grosse, R. Ranganath, and A. Y. Ng. Convolutional deep belief networks for scalable unsupervised learning of hierarchical representations. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 609–616. ACM, 2009.

[19] S. Lucas. ICDAR 2005 text locating competition results. In *Document Analysis and Recognition, 2005. Proceedings. Eighth International Conference on*, pages 80–84. IEEE, 2005.

[20] F. Mamalet and C. Garcia. Simplifying convnets for fast learning. In *Artificial Neural Networks and Machine Learning–ICANN 2012*, pages 58–65. Springer, 2012.

[21] M. Mathieu, M. Henaff, and Y. LeCun. Fast training of convolutional networks through ffts. *CoRR*, abs/1312.5851, 2013.

[22] L. Neumann and J. Matas. A method for text localization and recognition in real-world images. In *Proc. Asian Conf. on Computer Vision*, pages 770–783. Springer, 2010.

[23] L. Neumann and J. Matas. Text localization in real-world images using efficiently pruned exhaustive search. In *Proc. ICDAR*, pages 687–691. IEEE, 2011.

[24] L. Neumann and J. Matas. Real-time scene text localization and recognition. In *Proc. CVPR*, volume 3, pages 1187–1190. IEEE, 2012.

[25] L. Neumann and J. Matas. Scene text localization and recognition with oriented stroke detection. In *2013 IEEE International Conference on Computer Vision (ICCV 2013)*, pages 97–104, California, US, December 2013. IEEE. ISBN 978-1-4799-2839-2. doi: 10.1109/ICCV.2013.19.

[26] M. Oquab, L. Bottou, I. Laptev, and J. Sivic. Learning and transferring mid-level image representations using convolutional neural networks. In *Computer Vision and Pattern Recognition (CVPR)*, 2014.

[27] I. Posner, P. Corke, and P. Newman. Using text-spotting to query the world. In *Proc. of the IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, 2010.

[28] T. Quack. *Large scale mining and retrieval of visual data in a multimodal context.* PhD thesis, ETH Zurich, 2009.

[29] A. S. Razavian, H. Azizpour, J. Sullivan, and S. Carlsson. Cnn features off-the-shelf: an astounding baseline for recognition. *arXiv preprint arXiv:1403.6382*, 2014.

[30] R. Rigamonti, M. A. Brown, and V. Lepetit. Are sparse representations really relevant for image classification? In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 1545–1552. IEEE, 2011.

[31] R. Rigamonti, A. Sironi, V. Lepetit, and P. Fua. Learning separable filters. In *Computer Vision and Pattern Recognition (CVPR), 2013 IEEE Conference on*, pages 2754–2761. IEEE, 2013.

[32] P. Sermanet, D. Eigen, X. Zhang, M. Mathieu, R. Fergus, and Y. LeCun. Overfeat: Integrated recognition, localization and detection using convolutional networks. *arXiv preprint arXiv:1312.6229*, 2013.

[33] A. Shahab, F. Shafait, and A. Dengel. ICDAR 2011 robust reading competition challenge 2: Reading text in scene images. In *Proc. ICDAR*, pages 1491–1496. IEEE, 2011.

[34] H. O. Song, S. Zickler, T. Althoff, R. Girshick, M. Fritz, C. Geyer, P. Felzenszwalb, and T. Darrell. Sparselet models for efficient multiclass object detection. In *Computer Vision–ECCV 2012*, pages 802–815. Springer, 2012.

[35] H. O. Song, T. Darrell, and R. B. Girshick. Discriminatively activated sparselets. In *Proceedings of the 30th International Conference on Machine Learning (ICML-13)*, pages 196–204, 2013.

[36] Y. Taigman, M. Yang, M. Ranzato, and L. Wolf. Deep-Face: Closing the gap to human-level performance in face verification. In *IEEE CVPR*, 2014.

[37] A. Toshev and C. Szegedy. DeepPose: Human pose estimation via deep neural networks. *arXiv preprint arXiv:1312.4659*, 2013.

[38] K. van de Sande, J. Uijlings, T. Gevers, and A. Smeulders. Segmentation as selective search for object recognition. In *Computer Vision (ICCV), 2011 IEEE International Conference on*, pages 1879–1886. IEEE, 2011.

[39] V. Vanhoucke, A. Senior, and M. Z. Mao. Improving the speed of neural networks on cpus. In *Proc. Deep Learning and Unsupervised Feature Learning NIPS Workshop*, 2011.

[40] K. Wang, B. Babenko, and S. Belongie. End-to-end scene text recognition. In *Proc. ICCV*, pages 1457–1464. IEEE, 2011.

[41] T. Wang, D. J. Wu, A. Coates, and A. Y. Ng. End-to-end text recognition with convolutional neural networks. In *Pattern Recognition (ICPR), 2012 21st International Conference on*, pages 3304–3308. IEEE, 2012.

[42] H. Yang, B. Quehl, and H. Sack. A framework for improved video text detection and recognition. In *Int. Journal of Multimedia Tools and Applications (MTAP)*, 2012.